

Homework Project

Spring Boot — Filters, Security & Exception Handling

Fintech Backend Engineering Training | PostgreSQL Edition

Project Overview

You will build **WalletAPI**, a simplified fintech wallet backend using Spring Boot and PostgreSQL. The project is scoped to develop deep understanding of three backend infrastructure concerns: how requests are intercepted and logged (filters), who can access what (Spring Security + JWT), and how errors are communicated consistently (global exception handling).

Topic	Coverage	Weight
Spring Filters	Request logging filter, IP allowlist filter, filter chain ordering	30%
Spring Security	JWT authentication, role-based authorization, SecurityFilterChain	40%
Exception Handling	@ControllerAdvice, custom exceptions, RFC 7807 error responses	20%
Database & JPA	PostgreSQL schema, JPA entities, Spring Data repositories	10%

API Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/login	Public	Accepts credentials, returns JWT
GET	/api/health	Public	Health check, no auth required
GET	/api/wallet/balance	USER, ADMIN	Returns authenticated user's balance
GET	/api/wallet/transactions	USER, ADMIN	Returns last 10 transactions for the user
GET	/api/admin/users	ADMIN only	Returns list of all users

Part 1 — Database Schema & Setup

Your PostgreSQL database must contain the following four tables. Create a database named **walletdb** and apply the DDL below. You will manage schema changes using **Flyway** — place migration scripts in `src/main/resources/db/migration/` following the naming convention `V1__description.sql`.

users — stores application accounts

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
username	VARCHAR(50)	NOT NULL, UNIQUE
password	VARCHAR(255)	NOT NULL -- BCrypt hash
email	VARCHAR(100)	NOT NULL, UNIQUE
role	VARCHAR(20)	NOT NULL -- 'USER' or 'ADMIN'
is_active	BOOLEAN	NOT NULL DEFAULT TRUE
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

wallets — one wallet per user

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
user_id	BIGINT	NOT NULL, UNIQUE, FK -> users(id)
balance	NUMERIC(18,4)	NOT NULL DEFAULT 0.0000
currency	VARCHAR(3)	NOT NULL DEFAULT 'USD'
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

transactions — immutable ledger entries

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
wallet_id	BIGINT	NOT NULL, FK -> wallets(id)
amount	NUMERIC(18,4)	NOT NULL -- negative = debit, positive = credit
type	VARCHAR(20)	NOT NULL -- 'CREDIT' or 'DEBIT'
description	VARCHAR(255)	NULL
reference_id	VARCHAR(100)	NULL, UNIQUE -- external reference
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

request_logs — written by the logging filter

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
method	VARCHAR(10)	NOT NULL
uri	VARCHAR(500)	NOT NULL
status	INT	NULL -- populated after response
remote_ip	VARCHAR(50)	NOT NULL
username	VARCHAR(50)	NULL -- populated if authenticated
duration_ms	BIGINT	NULL
requested_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

Important notes on schema design:

- transactions is an immutable ledger — rows are INSERT only, never UPDATE or DELETE. This is a core fintech pattern.
- balance in wallets is a derived value kept in sync by your service layer. Do not recompute it by summing transactions on every request.
- NUMERIC(18,4) is used for all monetary values. Never use FLOAT or DOUBLE for money.
- All timestamps use TIMESTAMPTZ (timestamp with time zone) so the database stores UTC consistently.
- request_logs is written by your filter, not your service layer — keep this concern separate.

V1_initial_schema.sql (place in src/main/resources/db/migration/)

```
CREATE TABLE users (  
  id BIGSERIAL PRIMARY KEY,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  role VARCHAR(20) NOT NULL CHECK (role IN ('USER', 'ADMIN')),  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE wallets (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT NOT NULL UNIQUE REFERENCES users(id),  
  balance NUMERIC(18,4) NOT NULL DEFAULT 0.0000,  
  currency VARCHAR(3) NOT NULL DEFAULT 'USD',  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE transactions (  
  id BIGSERIAL PRIMARY KEY,  
  wallet_id BIGINT NOT NULL REFERENCES wallets(id),  
  amount NUMERIC(18,4) NOT NULL,  
  type VARCHAR(20) NOT NULL CHECK (type IN ('CREDIT', 'DEBIT')),  
  description VARCHAR(255),  
  reference_id VARCHAR(100) UNIQUE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE request_logs (  
  id BIGSERIAL PRIMARY KEY,  
  method VARCHAR(10) NOT NULL,  
  uri VARCHAR(500) NOT NULL,  
  status INT,  
  remote_ip VARCHAR(50) NOT NULL,  
  username VARCHAR(50),  
  duration_ms BIGINT,  
  requested_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

```
);
```

V2_seed_data.sql — two users with BCrypt-hashed passwords

```
-- Password for alice = 'password123'
-- Password for admin = 'admin456'
-- Hashes below are pre-computed BCrypt strings (cost factor 10)

INSERT INTO users (username, password, email, role) VALUES

('alice', '$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhwy', 'alice@example.com',
'USER'),

('admin', '$2a$10$GRLdNijSQMUvL/au9ofL.eDwmoohzzS7.rmNSJZ.0Fx0/BTk76klw', 'admin@example.com',
'ADMIN');

INSERT INTO wallets (user_id, balance, currency) VALUES

(1, 1500.0000, 'USD'),

(2, 9999.0000, 'USD');

INSERT INTO transactions (wallet_id, amount, type, description) VALUES

(1, 500.0000, 'CREDIT', 'Initial deposit'),

(1, 1000.0000, 'CREDIT', 'Salary payment'),

(1, -250.00, 'DEBIT', 'Utility bill'),

(2, 9999.0000, 'CREDIT', 'Admin seed balance');
```

Part 2 — application.properties Template

Use the template below as your starting configuration. Replace placeholder values with your own. Never commit real secrets to version control — for this homework using a local `.env` or hardcoded values in properties is acceptable, but note this in your README.

```
src/main/resources/application.properties

# Application
spring.application.name=walletapi
server.port=8080

# PostgreSQL
spring.datasource.url=jdbc:postgresql://localhost:5432/walletdb
spring.datasource.username=YOUR_DB_USER
spring.datasource.password=YOUR_DB_PASSWORD
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false

# Flyway
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration

# JWT
security.jwt.secret=REPLACE_WITH_A_256_BIT_BASE64_SECRET_KEY
security.jwt.expiry-ms=900000

# IP Allowlist
security.allowed-ips=127.0.0.1,0:0:0:0:0:0:1

# Logging
logging.level.com.example.walletapi=DEBUG
logging.level.org.springframework.security=INFO
```

ddl-auto=validate is intentional — it tells Hibernate to validate the schema against your entities but not modify it. Flyway owns all schema changes. If you see a schema validation error on startup, your entity mappings do not match the migration scripts.

Part 3 — Spring Filters

Implement the two filters below by extending **OncePerRequestFilter**. Register them as beans in a **FilterConfig** class using **FilterRegistrationBean** with explicit ordering.

1 Request Logging Filter (order = 1)

- On every incoming request: capture the HTTP method, URI, remote IP, and start timestamp.
- After the chain executes: capture the HTTP response status and compute duration in milliseconds.
- Persist a RequestLog entity to the request_logs table via RequestLogService.
- Also log to SLF4J at INFO level in the format: [METHOD] /uri -> STATUS (Xms) from IP.
- If the request has a valid JWT, extract the username from the SecurityContext and populate the username column in request_logs.

Hint: Wrap the HttpServletResponse in a ContentCachingResponseWrapper before calling filterChain.doFilter() so you can read the status code after the response is written.

2 IP Allowlist Filter (order = 2)

- Read the allowed IP list from security.allowed-ips in application.properties using @Value.
- On every request: compare request.getRemoteAddr() against the allowlist.
- If the IP is not allowed: write a 403 JSON error response directly to HttpServletResponse and return without calling filterChain.doFilter().
- The 403 JSON body must follow the standard ErrorResponse format defined in Part 6.
- This filter must still allow requests to pass to the logging filter's post-chain logic — ensure the log is still written even for blocked requests.

Note: Because this filter runs after the logging filter, the log entry will be written regardless of whether the IP is blocked. This is the correct behaviour — you want a record of blocked attempts.

Written question: Explain in 2–3 sentences why filter exceptions cannot be caught by @ControllerAdvice, and describe the approach you used to return a consistent error response from the IP allowlist filter.

Part 4 — Spring Security & JWT

3 UserDetailsServiceImpl

- Implement UserDetailsService by loading users from the users table via UserRepository.
- Map the role column to a GrantedAuthority — prefix it with ROLE_ (e.g. ROLE_USER, ROLE_ADMIN).
- If the user is not found, throw UsernameNotFoundException.
- Check the is_active flag — if false, throw DisabledException.

4 JwtUtil — Token Generation & Validation

- Create a JwtUtil @Component with the following methods:
- generateToken(UserDetails user) — signs a JWT with HS256 using the secret from properties, sets expiry to security.jwt.expiry-ms.
- validateToken(String token) — returns true if the signature is valid and the token is not expired.
- extractUsername(String token) — parses and returns the subject claim.
- extractRole(String token) — parses and returns the role claim.
- Store the JWT secret as a Base64-decoded byte[] loaded once at startup via @PostConstruct.

Hint: Use `io.jsonwebtoken.Jwts.parserBuilder().setSigningKey(key).build()` from the `jjwt` library. Wrap parsing in a try/catch and throw `InvalidTokenException` on any `JwtException`.

5 POST /api/auth/login

- Accept a LoginRequest DTO with username (required, non-blank) and password (required, non-blank) fields annotated with @Valid constraints.
- Delegate authentication to `AuthenticationManager.authenticate()`.
- On success: generate a JWT and return it in a LoginResponse DTO with fields: token, username, role, expiresIn.
- On failure: `AuthenticationException` is thrown — let `GlobalExceptionHandler` return HTTP 401.
- This endpoint must be publicly accessible (no token required).

6 JwtAuthenticationFilter (order = 3, after security filters)

- Extract the token from the Authorization header — expect the format: Bearer .
- If no Authorization header is present, do nothing and call `filterChain.doFilter()`.
- If a token is present: validate it using `JwtUtil`. If invalid, throw `InvalidTokenException`.
- If valid: load the `UserDetails`, construct a `UsernamePasswordAuthenticationToken`, and set it on `SecurityContextHolder`.
- Register this filter to run before `UsernamePasswordAuthenticationFilter` in `SecurityFilterChain`.

7 SecurityFilterChain Configuration

- Disable CSRF and set session management to STATELESS.
- Permit without authentication: POST `/api/auth/login` and GET `/api/health`.
- Require ADMIN role for: `/api/admin/**`.
- Require authentication for all other requests.
- Configure a custom `AuthenticationEntryPoint` that returns a 401 JSON `ErrorResponse` (not the default Spring HTML page).
- Configure a custom `AccessDeniedHandler` that returns a 403 JSON `ErrorResponse`.

Part 5 — Global Exception Handling

All error responses from WalletAPI must follow the structure below. Define an **ErrorResponse** record (or class) and use it consistently across all handlers, filters, and security entry points.

ErrorResponse — standard error shape for all API errors

```
{
  "timestamp": "2024-11-01T10:23:45Z", // ISO-8601 UTC
  "status": 403, // HTTP status code
  "error": "Forbidden", // HTTP reason phrase
  "message": "Access denied.", // Human-readable detail
  "path": "/api/admin/users" // Request URI
}
```

8 Custom Exception Classes

- `IpNotAllowedException` extends `RuntimeException` — used by the IP allowlist filter (HTTP 403).
- `InvalidTokenException` extends `RuntimeException` — used by `JwtUtil` and the JWT filter (HTTP 401).
- `ResourceNotFoundException` extends `RuntimeException` — used for 404 cases (HTTP 404).
- `AuthenticationFailedException` extends `RuntimeException` — for bad login credentials (HTTP 401).
- Each exception should accept a `String` message in its constructor.

9 `@RestControllerAdvice` — `GlobalExceptionHandler`

- Handle `IpNotAllowedException` — return HTTP 403 `ErrorResponse`.
- Handle `InvalidTokenException` — return HTTP 401 `ErrorResponse`.
- Handle `ResourceNotFoundException` — return HTTP 404 `ErrorResponse`.
- Handle `AuthenticationFailedException` — return HTTP 401 `ErrorResponse`.
- Handle `MethodArgumentNotValidException` — return HTTP 400 `ErrorResponse`; list all field validation errors in the message field.
- Handle generic `Exception` as a fallback — return HTTP 500 with message 'An unexpected error occurred.' Do not expose the stack trace or internal exception message.
- All handlers must return `ResponseEntity`.

Part 6 — Manual Testing Checklist

Verify your implementation against each scenario using **curl** or **Postman**. Include screenshots or curl output for each in your submission.

#	Scenario	Action	Expected	Pass
1	Health check	GET /api/health — no token	HTTP 200	■
2	Login — valid (USER)	POST /api/auth/login with alice / password123	HTTP 200 + JWT in response body	■
3	Login — valid (ADMIN)	POST /api/auth/login with admin / admin456	HTTP 200 + JWT with ADMIN role	■
4	Login — wrong password	POST /api/auth/login with alice / wrongpass	HTTP 401 + ErrorResponse JSON	■
5	Login — missing field	POST /api/auth/login with empty username	HTTP 400 + validation errors in message	■
6	Balance — authenticated	GET /api/wallet/balance with alice's Bearer token	HTTP 200 + balance for alice's wallet	■
7	Balance — no token	GET /api/wallet/balance with no Authorization header	HTTP 401 + ErrorResponse JSON	■
8	Balance — expired token	GET /api/wallet/balance with an expired JWT	HTTP 401 + ErrorResponse JSON	■
9	Transactions	GET /api/wallet/transactions with alice's token	HTTP 200 + list of up to 10 transactions	■
10	Admin — wrong role	GET /api/admin/users with alice's token (USER role)	HTTP 403 + ErrorResponse JSON	■
11	Admin — correct role	GET /api/admin/users with admin's token	HTTP 200 + list of users	■
12	IP block	Remove 127.0.0.1 from allowlist, retry any request	HTTP 403 + ErrorResponse JSON from filter	■
13	Request log written	After any request, query SELECT * FROM request_logs	Row present with correct method, URI, status, IP	■
14	Log includes username	Make authenticated request, check request_logs	username column populated correctly	■

Part 7 — Grading Rubric

Total: 100 points. Each criterion is assessed on whether it is fully implemented and correctly behaves against the test scenarios.

Criterion	What we look for	Pts
Database & Flyway	Schema matches DDL, Flyway migrations run cleanly, entities map correctly, seed data loads	10
Request logging filter	Logs to SLF4J, persists RequestLog to DB, captures status and duration, populates username when authenticated	12
IP allowlist filter	Reads from properties, blocks correctly with 403 JSON, does not break logging, filter order is correct	12
JWT generation (login)	Returns signed token, token contains correct claims, wrong credentials return 401 ErrorResponse	12
JWT authentication	Extracts and validates token, sets SecurityContext, expired/invalid tokens return 401 ErrorResponse, missing token falls through	15
SecurityFilterChain	Correct endpoint permissions, STATELESS sessions, custom 401/403 handlers return JSON ErrorResponse not HTML	13
Exception handling	All custom exceptions handled, consistent ErrorResponse shape, fallback handler hides internals, validation errors return 400	16
Written answers	Filter/ControllerAdvice boundary explanation is accurate and clear; filter ordering rationale is correct	6
Code quality	Clean package structure, no business logic in filters/controllers, no plain-text passwords, README present	4
Total		100

Common Pitfalls

- Exceptions from filters are not caught by `@ControllerAdvice` — write the error response directly to `HttpServletResponse` in the filter.
- The JWT filter must not throw when the Authorization header is absent — simply skip and let Spring Security handle it.
- `ddl-auto=validate` will fail on startup if your `@Entity` fields do not exactly match the Flyway-created columns.

- NUMERIC(18,4) in Postgres maps to BigDecimal in Java — never use double or float for money fields.
- BCrypt hashes are not reversible — if you regenerate them, the seed data passwords will change. Keep the pre-computed hashes from V2__seed_data.sql.
- Filter order: logging (1) → IP allowlist (2) → JWT authentication (3) → Spring Security filters.